

HOCHSCHULE
HANNOVER
UNIVERSITY OF
APPLIED SCIENCES
AND ARTS

–
*Fakultät IV
Wirtschaft und
Informatik*

Convolution und ihre Anwendung in der digitalen Audiobearbeitung

Seminarfach: Audio DSP

Julian Mueller Tom Nir
21. Mai 2026



1 Grundlagen: Audio DSP

2 Convolution

3 Literatur



Sampling I

- Abtastung stetiger Signale $\Rightarrow$ Folge diskreter Amplitudenwerte

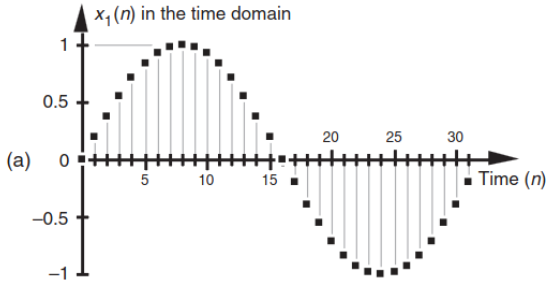


Abbildung: Horiz. Achse: Zeitbereich (n-ter Samplewert) Vert. Achse: Amplitude [1, Kap. 2]



Sampling II

- Berechnung von Signalwerten:

$$x[n] = \sin(2\pi \cdot f_0 \cdot n \cdot t_s) \quad (1)$$

Wobei:

- 2π : Eine komplette Schwingung (360° auf dem Einheitskreis)
- f_0 : Frequenz des Signals in Hz
- n : Sample-Index
- f_s : Sampling-Intervall (Zeit, die zwischen zwei Messungen vergeht)



Zeit- und Frequenzdomäne I

- Zeitdomäne: Horizontale -> diskrete Zeitindizes; Vertikale -> Amplitude
- Frequenzdomäne: Horizontale -> Frequenz in Hz ; Vertikale -> Magnitude

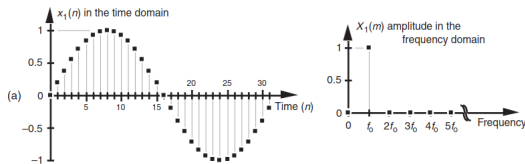


Abbildung: Gegenüberstellung: Zeit-/Frequenzdomäne einer Sinuswelle Lyons, 2011¹



Zeit- und Frequenzdomäne II

- Umwandlung eines Signals in Zeitdomäne in Frequenzdomäne mithilfe der *Fourier Transformation*:

$$X[m] = \sum_{n=0}^{N-1} x[n] \cdot e^{\frac{-i2\pi \cdot nm}{N}} \quad (2)$$

¹1, Kap. 2.



Nyquist-Shannon Grenze und Aliasing I

- Lt. Nyquist-Shannon-Theorem gilt:

$$x[n] = \sin(2\pi \cdot f_0 \cdot n \cdot t_s) = \sin(2\pi \cdot (f_0 + kf_s) \cdot nt_s) \quad (3)$$

Wobei: $k \in \mathbb{Z}$

- Daraus Folgt: Die Parameter f_0 und $(f_0 + kf_s)$ führen zum gleichen Amplitudenwert $\implies$ Nicht unterscheidbar
- Resultat: Mathematisch unendlich viele Aliase für jede Frequenz des Eingangssignals



Nyquist-Shannon Grenze und Aliasing II

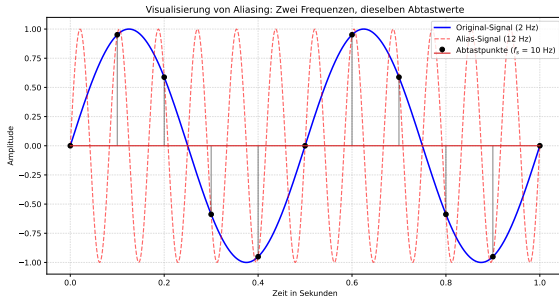


Abbildung: Für jedes Signal gibt es unendlich viele Interpretationsmöglichkeiten

Lyons, 2011²

²1, Kap. 2.



1 Grundlagen: Audio DSP

2 Convolution

3 Literatur



Laufzeitanalyse I

```
1 Algorithm directConvolution(inSignal, N, irSignal, M)
2   outLen <- N + M - 1      // O(1)
3   for i <- 0 to outLen - 1  // O(N + M + 1)
4     for k <- 0 to M - 1     // O(M)
5       if i-k < 0 or n <= i-k // O(1)
6         continue
7
8       outSignal[i] <- outSignal[i]
9       + irSignal[k] * inSignal[i-k] // Worst-Case: O(1)
10    done
11  done
12  return outSignal
```

Abbildung: Pseudocode: Implementierung der diskreten Convolution



Laufzeitanalyse II

- Worst-Case Laufzeitklasse:

$$\mathcal{O}(1 + (N + M - 1) \cdot (M + 2)) = \mathcal{O}(M^2 + NM - M + 2N - 1) \quad (4)$$

- ... vereinfacht sich lt. Polynomregel zu:

$$\mathcal{O}(M^2 + NM) \quad (5)$$

- ... es gilt aber

$$N \gg M \implies N \cdot M > M^2 \quad (6)$$

- ... woraus folgt: $\mathcal{O}(N \cdot M)$



Laufzeitanalyse III

```
1 Algorithm fftConvolution(inSignal, N, irSignal, M)
2   outLen <- N + M - 1 // O(1)
3   fftLen <- outLen    // O(1)
4
5   // zeroPad() füllt die Listen bis fftLen mit nullen auf
6   zeroPad(inSignal, fftLen) // O(n)
7   zeroPad(irSignal, fftLen) // O(n)
8
9   inSpectrum <- FFT(inSignal, N, fftLen) // O(n log(n))
10  irSpectrum <- FFT(irSignal, M, fftLen) // O(n log(n))
11
12  for f <- 0 to fftLen - 1 // Insg. O(n)
13    outSpectrum[f] <- inSpectrum[f] * irSpectrum[f]
14  done
15
16  outTimeDomain <- IFFT(outSpectrum, fftLen) // O(n log(n))
17
18  // real() isoliert den Realteil des komplexen IFFT-Outputs
19  for i <- 0 to outLen - 1 // Insg. O(n)
20    outSignal[i] <- real(outTime[i])
21  done
22
23  return outSignal
```

Abbildung: Pseudocode: Implementierung der Convolution mithilfe der *Fast Fourier Transformation*



Laufzeitanalyse IV

- Laufzeit:

$$\mathcal{O}(3(N \cdot \log(N)) + 4N + 2) \in \mathcal{O}(N \cdot \log(N)) \quad (7)$$



1 Grundlagen: Audio DSP

2 Convolution

3 Literatur



Literaturverzeichnis I

- [1] Richard G. Lyons. *Understanding Digital Signal Processing*. third. Pearson Education, 2011. ISBN: 978-0-13-702741-5.

